

Instituto Tecnológico®
de Aguascalientes

TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE AGUASCALIENTES

DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

PROGRAMACIÓN ORIENTADA A OBJETOS

Actividad: Simulación de Festival de Música

Tema: Herencia y Polimorfismo en Python

Alumno: Joaquín García

Carrera: Ingeniería en Sistemas Computacionales

Repositorio: https://github.com/joaquinrg132-png/festival-musica-poo

Docente: Profesor de Programación Orientada a Objetos

Aguascalientes, Ags. — Noviembre 2026

1. Objetivo de la actividad

Aplicar los conceptos de herencia y polimorfismo mediante la creación de un programa orientado a objetos en Python que simule un festival de música con diferentes tipos de artistas (Cantante, DJ, Banda), donde cada uno tenga su propia forma de presentarse y tocar.

2. Fundamento teórico

• Herencia: Se estructuró la clase base Artista con los atributos comunes nombre, genero y popularidad (1 al 100). Las clases derivadas Cantante, DJ y Banda reutilizan este constructor mediante super().__init__(...) y definen sus atributos específicos (cancion_mas_popular, estilo e integrantes).

• Polimorfismo: Cada subclase sobrescribe el método actuar(). La función iniciar_festival(lista_artistas) invoca a presentarse(), actuar() y despedirse() de forma directa sobre cada objeto, sin emplear condicionales para discriminar su tipo.

3. Enlace al repositorio de GitHub

Repositorio público: https://github.com/joaquinrg132-png/festival-musica-poo

4. Capturas de pantalla de la ejecución

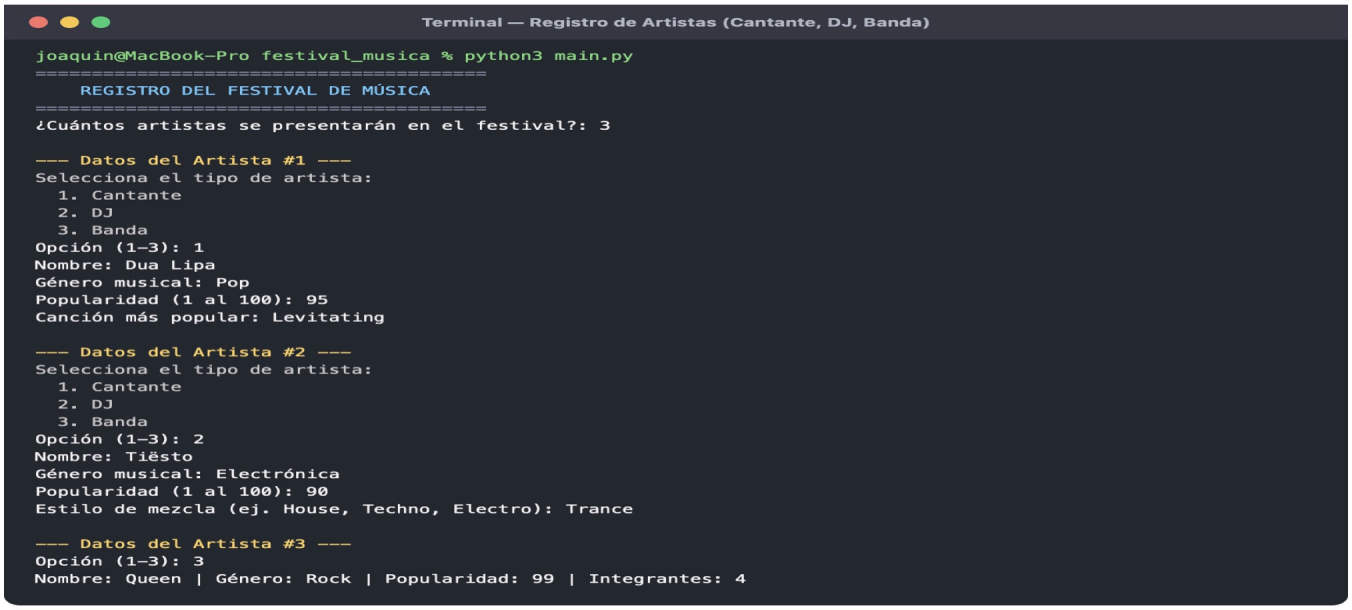


Figura 1: Registro interactivo de artistas solicitando atributos generales y específicos.

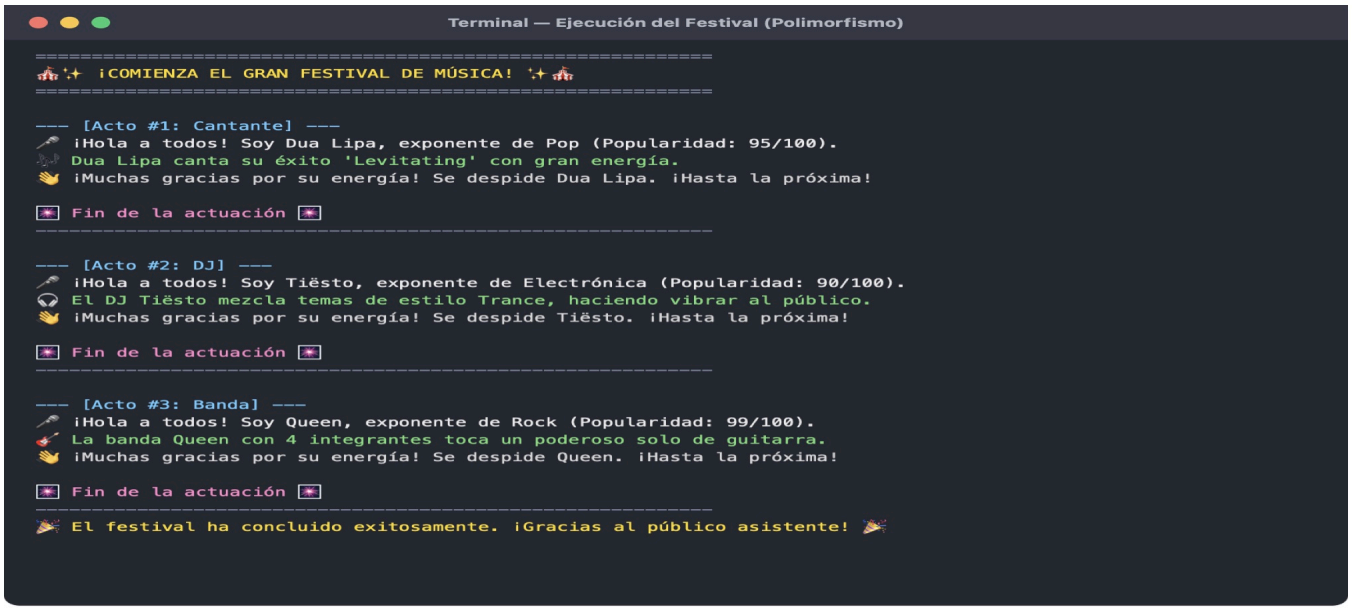


Figura 2: Ejecución del festival aplicando polimorfismo y separadores de actuación.

5. Código completo del programa (main.py) — Parte 1

```
class Artista:
    def __init__(self, nombre: str, genero: str, popularidad: int):
        self.nombre = nombre
        self.genero = genero
        self.popularidad = popularidad

    def presentarse(self):
        print(f"🎤 ¡Hola a todos! Soy {self.nombre}, exponente de {self.genero} (Popularidad: {self.pop

    def actuar(self):
        print(f"🎵 {self.nombre} está realizando una presentación en el escenario.")

    def despedirse(self):
        print(f"👋 ¡Muchas gracias por su energía! Se despide {self.nombre}. ¡Hasta la próxima!")

class Cantante(Artista):
    def __init__(self, nombre: str, genero: str, popularidad: int, cancion_mas_popular: str):
        super().__init__(nombre, genero, popularidad)
        self.cancion_mas_popular = cancion_mas_popular

    def actuar(self):
        print(f"🎶 {self.nombre} canta su éxito {self.cancion_mas_popular} con gran energía.")

class DJ(Artista):
    def __init__(self, nombre: str, genero: str, popularidad: int, estilo: str):
        super().__init__(nombre, genero, popularidad)
        self.estilo = estilo

    def actuar(self):
        print(f"🎧 El DJ {self.nombre} mezcla temas de estilo {self.estilo}, haciendo vibrar al público

class Banda(Artista):
    def __init__(self, nombre: str, genero: str, popularidad: int, integrantes: int):
        super().__init__(nombre, genero, popularidad)
        self.integrantes = integrantes

    def actuar(self):
        print(f"🎸 La banda {self.nombre} con {self.integrantes} integrantes toca un poderoso solo de g
```

5. Código completo del programa (main.py) — Parte 2

```
def iniciar_festival(lista_artistas):
    print("\n" + "=" * 60)
    print("🎪✨ ¡COMIENZA EL GRAN FESTIVAL DE MÚSICA! ✨🎪")
    print("=" * 60 + "\n")
    for i, artista in enumerate(lista_artistas, 1):
        print(f"--- [Acto #{i}] ---")
        artista.presentarse()
        artista.actuar()
        artista.despedirse()
        print("\n🎆 Fin de la actuación 🎆\n" + "-" * 60 + "\n")

def pedir_entero_rango(mensaje: str, minimo: int, maximo: int) -> int:
    while True:
        try:
            valor = int(input(mensaje))
            if minimo <= valor <= maximo: return valor
            print(f"⚠️ Por favor ingresa un número entre {minimo} y {maximo}.")
        except ValueError:
            print("⚠️ Entrada no válida. Ingresa un número entero.")

def main():
    num = int(input("¿Cuántos artistas se presentarán en el festival?: "))
    lista_artistas = []
    for i in range(1, num + 1):
        tipo = pedir_entero_rango("Opción (1. Cantante, 2. DJ, 3. Banda): ", 1, 3)
        nom = input("Nombre: ").strip()
        gen = input("Género musical: ").strip()
        pop = pedir_entero_rango("Popularidad (1 al 100): ", 1, 100)
        if tipo == 1:
            lista_artistas.append(Cantante(nom, gen, pop, input("Canción más popular: ")))
        elif tipo == 2:
            lista_artistas.append(DJ(nom, gen, pop, input("Estilo de mezcla: ")))
        else:
            lista_artistas.append(Banda(nom, gen, pop, pedir_entero_rango("Integrantes: ", 1, 100)))
    iniciar_festival(lista_artistas)

if __name__ == "__main__":
    main()
```

6. Conclusión

El desarrollo de este programa permitió constatar de forma práctica la utilidad de la herencia para evitar la redundancia de código, centralizando los atributos y métodos comunes en la clase padre Artista. A su vez, el polimorfismo hizo posible que la función iniciar_festival ejecute el método actuar() de cada objeto según su clase específica, sin requerir condicionales ni acoplamiento, logrando una arquitectura modular, escalable y mantenible.